SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

By: hackathon4dummies

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces "Gold Digger AI Assistant" as a strategic solution designed to address emotional decision-making in gold investment and the confusion caused by constantly fluctuating gold prices.

Instead of relying only on raw charts or static price displays, the system integrates real-time data with AI-driven analysis to help users make informed, rational, and personalized decisions.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the Gold Digger development.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*

   The system is designed as a cloud-based web platform with a modular structure that separates the frontend, backend, and AI processing components. This separation allows each part of the system to operate independently, improving scalability and maintainability.

   The frontend serves as the user interface, presenting gold price data, charts, and insights in a clear dashboard format. The backend acts as the intermediary layer that handles data requests, processes inputs, and communicates with external services. An AI Service Layer, powered by the Z.AI GLM, is integrated to perform advanced analysis and generate intelligent recommendations.

   The architecture consists of the following key components:

   - *A responsive frontend dashboard for visualization and interaction*
   - *Backend API services for data handling and system logic*
   - *An AI Service Layer for decision intelligence processing*
   - *External APIs that provide real-time gold price data*

2. ***Data Flows:***

   The system processes two main streams of data: real-time market data and user-provided inputs. Gold price data is continuously retrieved from external APIs and sent to the backend for formatting and validation. At the same time, users interact with the system by entering inputs such as budget, investment goals, or specific queries.

© UMHackathon 2026

These inputs are combined with the market data and passed to the AI model for analysis. The processed results are then returned to the frontend, where they are displayed as charts, insights, and recommendations.

The overall flow can be summarized as:

- *Gold price data → Backend → AI processing*
- *User input → Backend → AI processing*
- *AI output → Backend → Frontend dashboard*

This structured flow ensures that users receive updated and relevant information in near real-time.

3. ***Model Process:***

The model process represents the end-to-end workflow of how the system generates decision support. It begins with the collection of real-time gold price data alongside user-specific inputs. The AI model then analyzes this information to identify trends, measure volatility, and evaluate potential risks.

Based on this analysis, the system generates a recommendation such as Buy, Hold, or Wait. What makes the system more effective is its ability to provide an explanation in simple language, allowing users to understand not just the result, but also the reasoning behind it.

The process includes:

- *Data collection from APIs and user input*
- *Trend and pattern analysis*
- *AI-based reasoning using Z.AI GLM*
- *Generation of recommendation with explanation*

4. ***Role of Reference:***

This document serves as a technical reference for developers and testers involved in the project. It provides a clear overview of the system architecture, data flow, and model logic, ensuring that all components are implemented consistently.

In addition, it supports the integration of the Z.AI GLM as the core decision intelligence engine and helps guide future improvements, testing processes, and system scalability.

1.2. Background:

Traditionally, users rely on raw price charts or static gold prices provided by physical shops or basic online platforms when making investment decisions. While these sources provide access to market

© UMHackathon 2026

data, they often lack meaningful interpretation and do not consider individual financial goals or risk preferences.

As a result, users may struggle to identify the right time to enter or exit the market, leading to emotionally driven decisions and potential financial losses. This highlights the need for a more intelligent system that not only presents data but also helps users understand and act on it effectively.

1. ***Previous Version:***

   Existing solutions in the market typically function as simple price display platforms or rely on manual tracking through social media and informal sources. These approaches are limited in their ability to provide insights, as they do not actively analyze trends or personalize information for users.

   Gold Digger AI Assistant improves upon these limitations by transforming the system from a passive "display-only" platform into an active Decision Intelligence engine. Instead of merely showing gold prices, the system interprets data, evaluates market conditions, and provides actionable recommendations.

2. ***Changes in Major Architectural Components***

   One of the key enhancements in the system is the introduction of a dedicated AI Service Layer powered by the Z.AI GLM. This component enables the system to process both structured market data and unstructured user inputs, such as natural language queries.

   Unlike traditional gold-tracking tools, which primarily focus on data display, this architecture allows the system to perform deeper analysis, generate insights, and deliver personalized recommendations. The integration of AI also improves the system's ability to adapt to different user needs and scenarios.

3. ***New capabilities that are only introduced in this***

   The upgraded system introduces new capabilities that were not present in previous solutions, particularly in terms of intelligent analysis and user support.

   a. ***Explainable Strategy Generation:*** *The system translates complex market data into simple, human-readable insights. Instead of presenting raw numbers, it provides clear recommendations such as Buy, Hold, or Wait, along with concise explanations to help users understand the reasoning behind each decision.*

   b. ***Personalized Decision Intelligence:*** *The system tailors its recommendations based on user-specific inputs such as budget, risk tolerance, and investment goals. This allows*

© UMHackathon 2026

each user to receive guidance that is relevant to their financial situation rather than relying on generic market advice.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Individual Investors / Casual Savers	*Provide budget and goals to receive automated buying advice.*	*Simple, jargon-free "buy/wait" recommendations in plain language.*
Development Team	*Integrate LLM and external APIs while maintaining modularity.*	*Clear API usage contracts for Z.AI GLM and Gold Price API.*
QA Team	*Validate system behavior, especially LLM accuracy and edge cases.*	*Defined scope for response formats and coverage of API failure states.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based AI Assistant*
Architecture	*Cloud-Server with Microservices*

Gold Digger *is structured as a client-server-based system. The web clients communicate via a Backend API layer that orchestrates three independent microservices: Market Data Service, Strategy Reasoning Service, and Risk Alert Service. These services are designed for deployment on cloud infrastructure.*

2.1.2 LLM as Service Layer

In the Gold Digger AI Assistant, the Z.AI GLM is integrated as a dedicated AI Service Layer, rather than a generic AI module. This layer acts as the core decision intelligence engine, responsible for interpreting user input, analyzing market data, and generating explainable recommendations.

The GLM is accessed through API calls from the backend, where prompts are dynamically constructed using both real-time gold price data and user-specific inputs.
The following Dependency diagram includes:

2.1.3 Dependency Diagram

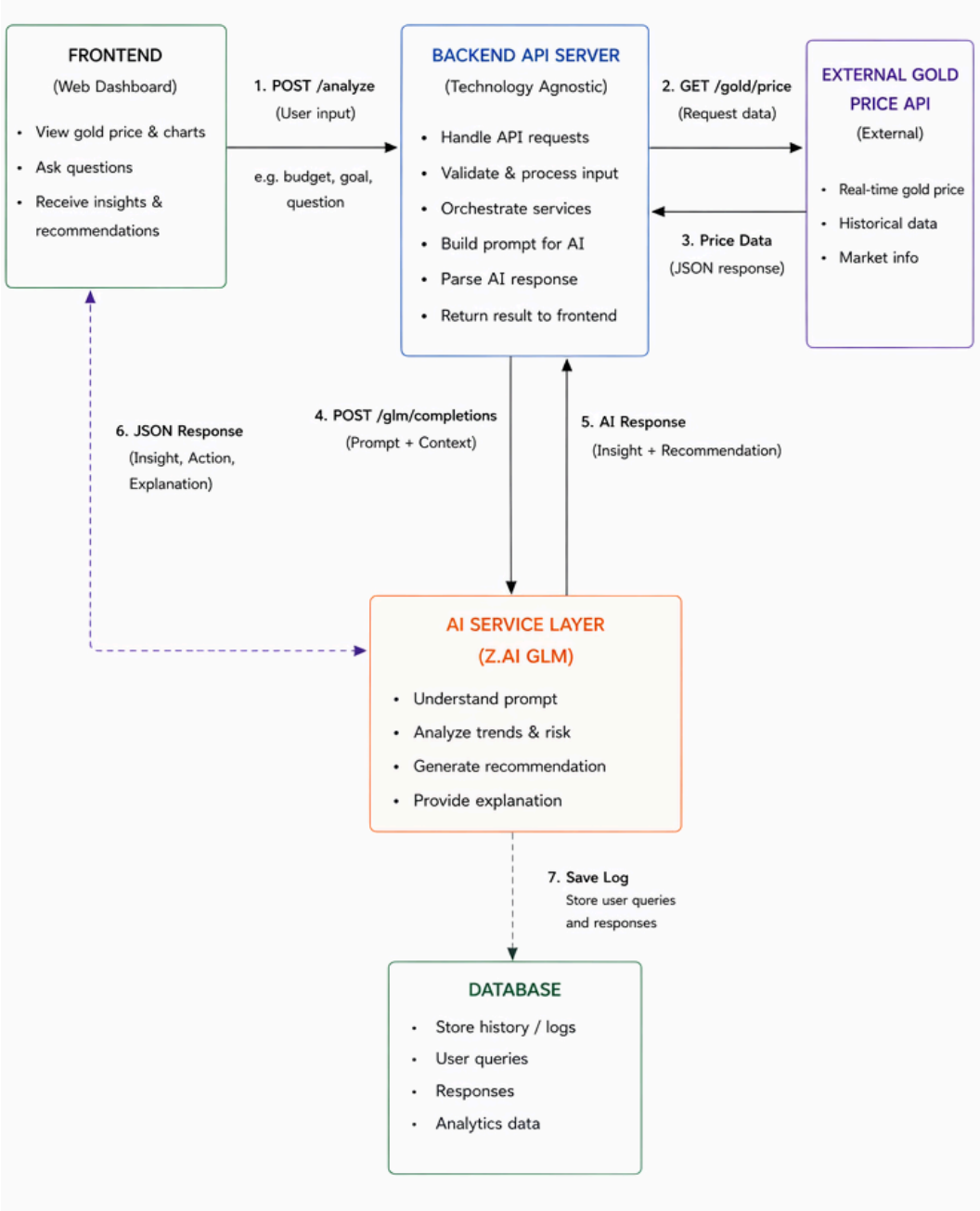
The system is structured into four main layers: the Frontend, Backend API Server, External Gold Price API, and the AI Service Layer powered by Z.AI GLM. The frontend serves as the user-facing interface where users view dashboards, charts, and AI-generated insights. All user interactions, such as submitting queries or investment inputs, are sent to the backend through API requests.

The backend acts as the central orchestration layer. It is responsible for handling incoming requests, validating user inputs, and coordinating communication between external services. When a request is received, the backend retrieves real-time gold price data from the external Gold Price API to ensure that analysis is based on up-to-date market conditions.

© UMHackathon 2026

After collecting the necessary data, the backend constructs a structured prompt that includes market information and user inputs. This prompt is then sent to the AI Service Layer (Z.AI GLM), which performs analysis, detects trends, evaluates risk levels, and generates investment recommendations.

Once the AI returns a response, the backend parses the output into a structured format and sends it back to the frontend. The frontend then displays the results in the form of insights, charts, and recommendation cards. Optionally, a database layer may store user interactions and results for logging or future analysis.



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram

The process begins when the user opens the application and accesses the dashboard through the frontend interface. The user then enters input such as investment budget or a natural language question like “Is it a good time to buy gold?”

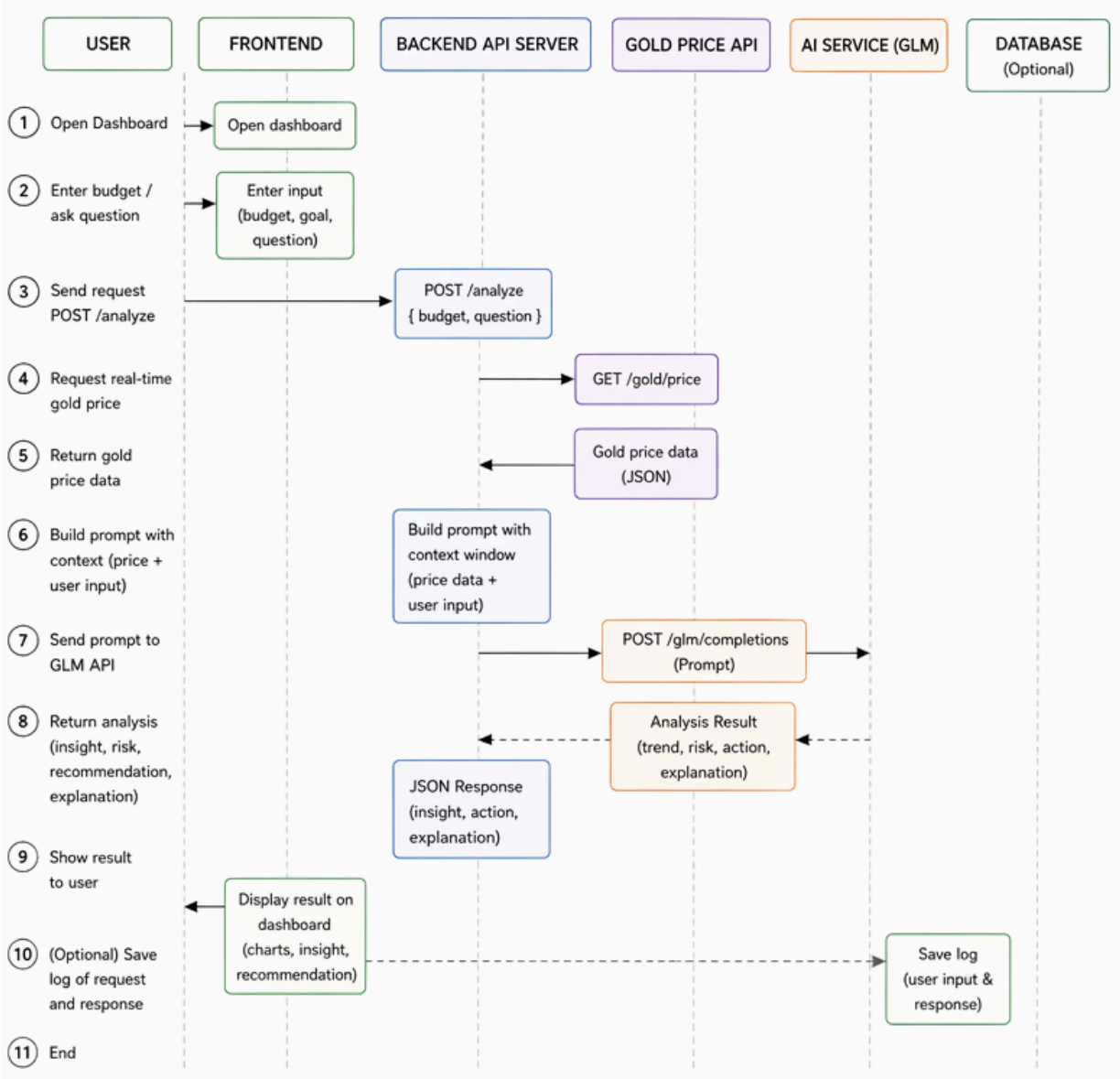
This input is sent from the frontend to the backend via an API request. Upon receiving the request, the backend retrieves real-time gold price data from the external Gold Price API to ensure the analysis reflects current market conditions.

Next, the backend combines the user input with market data to construct a structured prompt. This prompt is then sent to the Z.AI GLM, which processes the information by analyzing trends, evaluating volatility, and generating a recommendation.

The AI returns a response containing key outputs such as trend direction, risk level, suggested action (Buy, Hold, or Wait), and an explanation. The backend then formats this response into a structured JSON object and sends it back to the frontend.

Finally, the frontend displays the result to the user in the dashboard, including charts, insights, and recommendation cards. This complete flow ensures that the system delivers real-time, explainable, and personalized investment guidance in a seamless manner.

© UMHackathon 2026



2.2. *Technological Stack*

2.2.1. *Frontend*

Next.js : Used to build a responsive and interactive dashboard for the Gold Digger AI Assistant. The interface is designed to provide real-time gold market insights, AI-driven recommendations, and user-friendly investment tools. It focuses on a clean fintech-style experience that allows users to easily view gold prices, market trends, and decision suggestions in one place

2.2.2. *Backend*

Next.js API Routes + Z.ai GLM : The backend layer handles core application logic, including gold price analysis, Buy/Hold decision engine, and AI-powered Q&A. Z.ai GLM is integrated as the primary AI engine to generate intelligent responses, interpret market conditions, and

© UMHackathon 2026

provide contextual investment recommendations. This enables the system to function as an AI-assisted financial decision tool with real-time reasoning capabilities

2.2.3. Cloud/Deployment

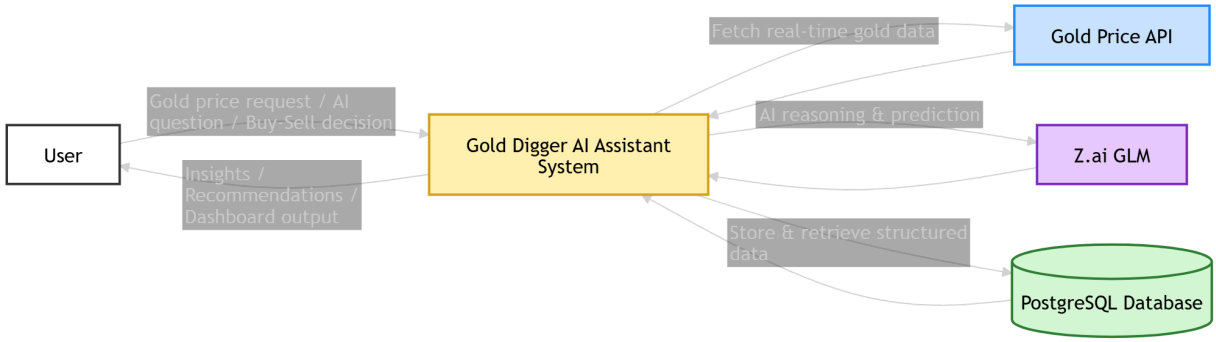
Vercel – Used to deploy the Gold Digger AI Assistant with high scalability and fast performance, supporting both frontend and backend functionalities in a serverless environment.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

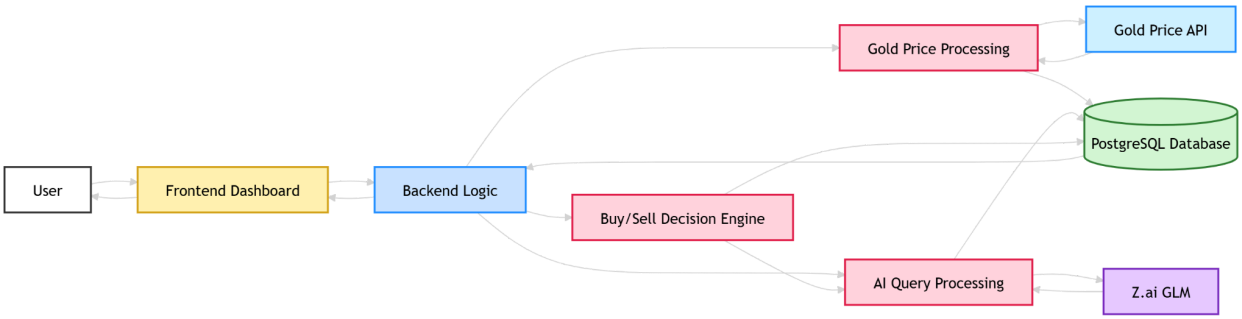
a. DFD - Level 0



The user interacts with the system by requesting gold prices, asking AI questions, or generating Buy/Hold investment decisions. The system acts as a central platform that processes all requests and returns insights and recommendations to the user.

The system communicates with external components including the Gold Price API for real-time market data, Z.ai GLM for AI-driven insights, and the PostgreSQL database for storing and retrieving structured data. This ensures all data flows are centralized, secure, and properly managed without direct user access to the database.

b. DFD - Level 1



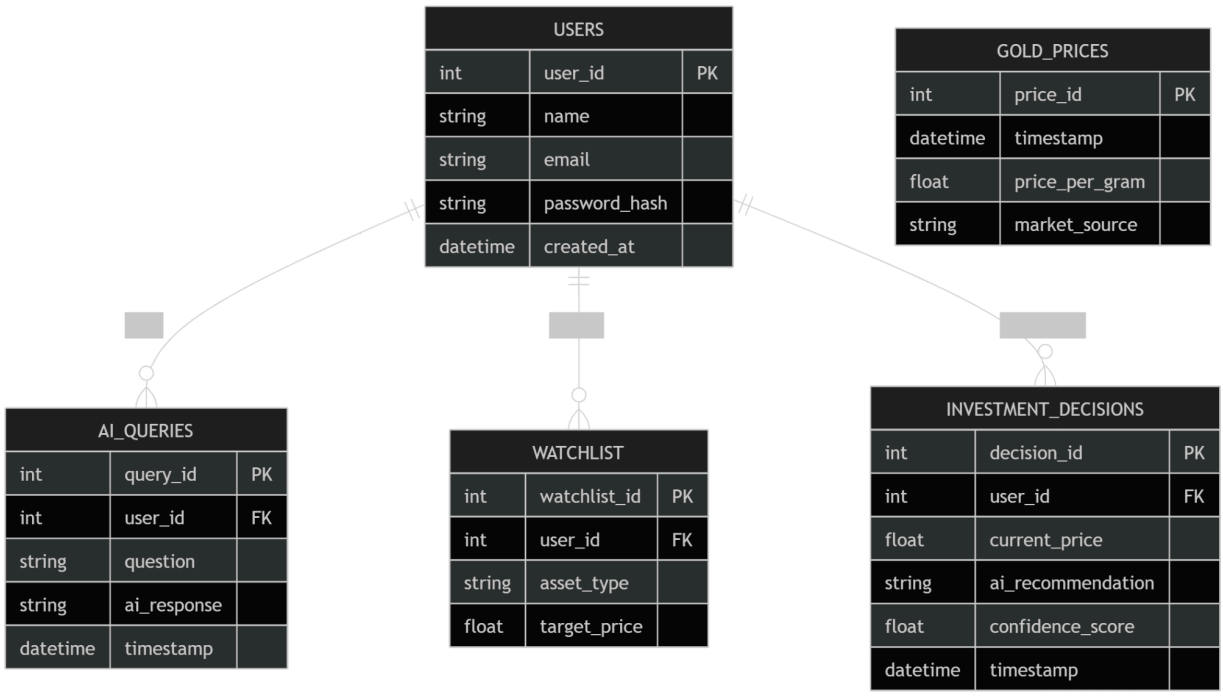
© UMHackathon 2026

The frontend receives user input and forwards it to the backend, which distributes it into three main processes: gold price processing, AI query processing, and Buy/Hold decision generation. Each process handles a specific function to ensure efficient and organized data handling.

Processed results are either stored in the PostgreSQL database or sent back to the frontend for user display. This structure ensures smooth communication between components, proper separation of logic, and secure handling of external APIs and AI services while maintaining real-time responsiveness.

2.3.2. **Normalized Database Schema**

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships.



3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. The following are the core features that the team will implement and demonstrate by the end of UMHackathon 2026. These features represent the essential functionality of the Gold Digger AI Assistant as a Decision Intelligence System.

© UMHackathon 2026

#	Features	Description
1	Real-Time Gold Price Dashboard	Displays live gold price updates fetched from external Gold Price APIs. The dashboard includes current price, percentage change, and visual indicators of market movement.
2	AI Investment Recommendation Engine (Z.AI GLM)	Processes user inputs (e.g., budget, questions) and combines them with real-time market data to generate Buy, Hold, or Wait recommendations with explanations.
3	Explainable AI Insight Panel	Displays human-readable explanations behind each recommendation, including trend summary, risk level, and reasoning in simple language.
4	Basic Decision Input Tool	Allows users to input their budget or investment intent and receive a quick AI-generated suggestion based on current market conditions.

3.2. Non-Functional Requirements (NFRs)

The following non-functional requirements define the system quality attributes that ensure performance, reliability, and scalability of the Gold Digger AI Assistant.

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	The system must handle multiple concurrent users accessing real-time gold data and AI analysis during peak usage periods.	Use modular cloud-based architecture with scalable backend services and load balancing to distribute API requests efficiently.
Reliability	The system must ensure consistent availability of gold price data and AI recommendations, minimizing downtime and preventing incomplete transactions or failed responses.	Implement API error handling, retry mechanisms, and fallback responses when external services are unavailable.
Maintainability	System components such as frontend, backend, and AI service layer should be modular and independently maintainable.	Use separated service architecture with clear API boundaries and independent deployment pipelines.

© UMHackathon 2026

Token Latency	*AI-generated responses from Z.AI GLM must be returned within acceptable response time (e.g., under 3–5 seconds under normal load).*	*Optimize prompt size, use structured context injection, and implement loading states with asynchronous API handling.*
Cost Efficiency	*API usage and token consumption must be optimized to avoid unnecessary cost overhead.*	*Limit prompt length, use cached responses for repeated queries, and preprocess data before sending to GLM.*

3.3. Out of Scope / Future Enhancements

The following features are considered part of the long-term vision of the Gold Digger AI Assistant but will not be implemented within the UMHackathon 2026 timeframe.

a. ***Advanced Portfolio Tracking System:*** *Allow users to track multiple investments and view overall performance analytics.*

b. ***Personalized Financial Profiles:*** *AI-driven user profiling based on risk behavior and investment history.*

c. ***What-If Simulation Engine (Advanced Version):*** *Enhanced simulation for long-term forecasting and multi-scenario comparison.*

d. ***Market News Integration Module:*** *Real-time integration of global financial news affecting gold prices.*

e. ***Social / Community Insight Feature:*** *Allow users to view market sentiment or crowd-based predictions.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

To ensure stable deployment in the Gold Digger AI Assistant, the system uses a grayscale rollout strategy. New features such as gold price insights, buy/hold recommendations, or AI Q&A improvements are first released to a small user group (around 5%). During this phase, key metrics like AI response accuracy, latency, API stability, and user engagement are monitored. If no critical issues occur, the rollout is gradually expanded until it reaches 100% of users.

© UMHackathon 2026

After full deployment, A/B testing is used to compare different AI versions. Users are split into groups exposed to different variants, such as:

- *Different recommendation logic (conservative vs aggressive)*
- *Different prompt styles for AI responses*

Performance is evaluated based on engagement, trust in recommendations, and decision outcomes. The best-performing version is selected as the default production model.

4.1.2. Emergency Rollback (ER) & Golden Release

The system maintains a Golden Release, which is the most stable and verified version of the Gold Digger AI Assistant. It includes tested AI models, stable gold price API integration, and reliable recommendation logic. This version acts as the fallback baseline in production to ensure system safety.

If critical issues occur such as incorrect AI recommendations, failure in retrieving live gold prices, high latency, or system instability, an Emergency Rollback (ER) is triggered. The system automatically reverts to the Golden Release through an automated deployment pipeline. This ensures users continue receiving stable and reliable outputs with minimal disruption.

4.1.3. Priority Matrix:

4.1.3.1. The Priority Matrix defines how system issues in the Gold Digger AI Assistant are classified based on severity and business impact. It ensures that critical issues affecting financial recommendations or system stability are handled immediately, while less urgent issues are managed in a structured development backlog.

a. P1 – Critical

This level is triggered when core system functionality is severely affected, especially features related to AI recommendations or live gold price retrieval. Examples include incorrect buy/hold signals, gold price API failure rate above 1%, or complete system downtime. In this case, an Emergency Rollback is immediately executed to restore the system to the Golden Release version.

b. P2 – High

This is triggered when the system is still operational but performance is significantly affected, such as high latency in AI responses or error rates between 0.3% to 1%. The team will apply mitigation actions such as disabling affected features, throttling requests, or performing partial rollback while investigating the issue.

c. P3 – Medium

© UMHackathon 2026

This includes non-critical issues such as minor bugs in UI, small inconsistencies in AI responses, or non-blocking API errors. These issues do not affect core functionality and are logged for fixing in the next development cycle.

d. P4 – Low

This covers minor improvements, UI enhancements, or feature suggestions that do not impact system performance or user decision-making. These are reviewed and prioritized during future sprint planning based on available resources.

4.2. Monitoring

4.2.1. The system includes a monitoring plan to ensure continuous performance, reliability, and early detection of issues across all components of the Gold Digger AI Assistant. Monitoring focuses on API performance, system health, and AI response quality to maintain a smooth user experience.

The backend API server will track key metrics such as request response time, error rates, and system uptime. Any abnormal behavior, such as slow responses or failed API calls, will trigger alerts for further investigation. In particular, calls to external services such as the Gold Price API and Z.AI GLM are closely monitored, as they are critical to system functionality.

To maintain system health, the following monitoring mechanisms are implemented:

- ***API Performance Monitoring:*** *Tracks latency and success rates of backend, Gold Price API, and GLM API calls. Alerts are triggered if response time exceeds a defined threshold (e.g., 3–5 seconds).*
- ***Error and Failure Detection:*** *Detects failed API calls, invalid responses, or system exceptions. Automatic fallback responses are provided to users when errors occur.*
- ***AI Response Validation:*** *Ensures that outputs from the Z.AI GLM follow the expected format (trend, risk level, recommendation, explanation). Any abnormal or incomplete response is flagged.*
- ***System Health Monitoring:*** *Monitors server availability and uptime. Alerts are triggered if services become unavailable or unstable.*
- ***Logging and Audit Trails:*** *Stores logs of user requests, API calls, and AI responses to support debugging and performance analysis.*

4.3. Assumptions:

The following assumptions are made regarding the operational and environmental conditions during the development and deployment of the Gold Digger AI Assistant:

a. Users have stable internet connectivity when accessing the web dashboard and submitting queries.

b. External APIs (Gold Price API and Z.AI GLM) are available and responsive under normal conditions.

© UMHackathon 2026

c. *Users provide valid and reasonable inputs (e.g., realistic budget values and meaningful queries).*
d. *The system operates in a standard web browser environment without requiring specialized hardware.*
e. *Real-time gold price data provided by external APIs is accurate and up-to-date.*
f. *The AI model (Z.AI GLM) returns responses within acceptable latency limits under normal load conditions.*

4.4. External Dependencies:

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output*	*High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]*
Gold Price API	*Tracking rider location and optimize the route*	*Medium level- if real time tracking is being disabled.*

5. Project Management & Team Contributions

5.1. Project Timeline

Day	Phase	Justification
Day 1-2	Project Planning and System Design	Problem scope, target users, and core features are defined to ensure alignment with the requirements.
Day 3-5	Data and Z.AI GLM Integration	The Z.AI API and GLM are integrated as the core decision engine.
Day 6-7	Frontend Development	The user interface is developed and connected to backend services.
Day 8-9	Testing and Validation	The system is tested using simulated scenarios to validate decision quality.

© UMHackathon 2026

Day 10	Demo Preparation and Documentation Finalization	Final demo flow is prepared to clearly showcase the AI-driven decision intelligence and all of the required documents are finalized.

5.2. *Team Members Role*

Team Member	Roles	Responsibilities
Norlydia	GLM engineer	• Design and refine prompts for Z.AI GLM • Structure input context
Nur Alya	Frontend Developer	• Build user interface • Design user interaction
Alia Amirah	Data and Simulation Engineer	• Simulate different market scenarios • Ensure data consistency and correctness
Siti Nursyahirah	Project Manager	• Define scope • Coordinate team progress and timeline

5.3. *Recommendations*

Scalability

- Use cloud deployment to handle more users when needed
- Separate system into components (UI, data, GLM) so each part can scale independently

Performance

- Optimize GLM prompts to reduce response time and cost
- Use caching for repeated data like gold prices or common queries
- Use asynchronous calls to avoid delays when waiting for GLM response

Reliability

- Add fallback response if GLM fails (e.g., simple rule-based suggestion)
- Implement timeout & retry for API calls
- Validate user input to avoid incorrect outputs

© UMHackathon 2026